
BookMyShow

Database Design & Architecture

Case Study

Prepared by: Naina Yadav
DBMS: MySQL 8.0+
Version: 1.0

1. Introduction & Business Context

BookMyShow is India's largest online entertainment ticketing platform, processing millions of ticket bookings daily across movies, live events, sports, and plays. This document presents the complete relational database design that powers the platform — covering schema design, entity relationships, foreign key constraints, indexing strategy, and referential integrity rules.

The database must satisfy the following core business requirements:

- Users can search for movies, shows, and events by city, language, genre, and date.
- Multiple venues exist per city; each venue has multiple screens; each screen runs multiple shows per day.
- Every screen has a fixed set of seats categorised by tier (Silver / Gold / Platinum) with different price multipliers.
- A booking locks one or more seats for a specific show and links to exactly one payment transaction.
- Promotional offers (coupon codes) can be optionally applied to a booking for a discount.
- Cancellations trigger a refund workflow updating booking and payment status.

2. System Architecture Overview

The platform follows a layered architecture. The database sits at the persistence layer, accessed by backend microservices over an ORM or JDBC connection pool.

Architecture layers:

- **Client layer:** Web browser / iOS / Android app
- **API Gateway:** Routes requests to appropriate microservices (Auth, Booking, Payment)
- **Microservices:** Search Service, Booking Service, Payment Service, Notification Service
- **Database layer:** MySQL 8.0 primary with read replicas for search queries
- **Cache layer:** Redis — caches seat availability per show, TTL = 10 minutes

The database schema is designed around **Third Normal Form (3NF)** to eliminate redundancy while maintaining referential integrity across all 12 tables.

3. Entity-Relationship Summary

The schema contains 12 tables across 3 logical domains:

Domain	Tables	Purpose
Location	CITIES, VENUES	Geographic data — where shows are held
Content	MOVIES, SCREENS, SHOWS, SEATS	What is being shown, where, and at what time
Transactions	USERS, BOOKINGS, BOOKING_SEATS, PAYMENTS, OFFERS, BOOKING_OFFERS	Who books, which seats, how they pay, and any discounts applied

4. Detailed Table Schemas

Each table is described below with column names, data types, constraints, and business purpose.

4.1 USERS

Column Name	Data Type	Constraint	Null	Description
user_id	INT	PK, AUTO_INCREMENT	NO	Unique user identifier
name	VARCHAR(100)		NO	Full name of the registered user
email	VARCHAR(150)	UNIQUE	NO	Login email — enforced unique
phone	VARCHAR(15)	UNIQUE	NO	Mobile number — unique per account
password_hash	VARCHAR(255)		NO	SHA-256 hashed password
created_at	TIMESTAMP	DEFAULT NOW()	NO	Account registration timestamp

4.2 CITIES

Column Name	Data Type	Constraint	Null	Description
city_id	INT	PK, AUTO_INCREMENT	NO	Unique city identifier
city_name	VARCHAR(80)		NO	City name e.g. Mumbai
state	VARCHAR(80)		NO	State name e.g. Maharashtra

4.3 VENUES

Column Name	Data Type	Constraint	Null	Description
venue_id	INT	PK, AUTO_INCREMENT	NO	Unique venue identifier
venue_name	VARCHAR(150)		NO	Cinema name e.g. PVR Juhu
address	VARCHAR(300)		NO	Full street address
city_id	INT	FK → CITIES	NO	Which city the venue is in
total_screens	INT	DEFAULT 1	NO	Number of screens in the venue

4.4 MOVIES

Column Name	Data Type	Constraint	Null	Description
movie_id	INT	PK, AUTO_INCREMENT	NO	Unique movie identifier
title	VARCHAR(200)		NO	Movie title
genre	VARCHAR(60)		NO	Genre e.g. Action, Comedy
language	VARCHAR(40)		NO	Language e.g. Hindi, English
duration_mins	INT		NO	Total runtime in minutes
certificate	VARCHAR(10)		NO	Censor rating: U, UA, A, S
release_date	DATE		NO	Theatrical release date

4.5 SCREENS

Column Name	Data Type	Constraint	Null	Description
screen_id	INT	PK, AUTO_INCREMENT	NO	Unique screen identifier
venue_id	INT	FK → VENUES	NO	Venue where the screen is located
screen_name	VARCHAR(60)		NO	e.g. Audi 1, IMAX Hall
total_seats	INT		NO	Physical seat count in screen
screen_type	VARCHAR(20)		NO	Technology: 2D, 3D, IMAX, 4DX

4.6 SHOWS

Column Name	Data Type	Constraint	Null	Description
show_id	INT	PK, AUTO_INCREMENT	NO	Unique show identifier
movie_id	INT	FK → MOVIES	NO	Which movie is being screened
screen_id	INT	FK → SCREENS	NO	Which screen hosts the show
show_time	DATETIME	UNIQUE(screen_id,time)	NO	Date and time of the show
base_price	DECIMAL(8,2)		NO	Base ticket price before multiplier
status	VARCHAR(20)	DEFAULT OPEN	NO	OPEN / HOUSEFUL / CANCELLED

4.7 SEATS

Column Name	Data Type	Constraint	Null	Description
seat_id	INT	PK, AUTO_INCREMENT	NO	Unique seat identifier
screen_id	INT	FK → SCREENS	NO	Screen where seat is located
row_label	CHAR(1)		NO	Row letter: A, B, C...
seat_number	INT		NO	Seat number within the row
category	VARCHAR(20)		NO	SILVER / GOLD / PLATINUM

Column Name	Data Type	Constraint	Null	Description
price_multiplier	DECIMAL(4,2)	DEFAULT 1.00	NO	Multiplied with show base_price

4.8 BOOKINGS

Column Name	Data Type	Constraint	Null	Description
booking_id	INT	PK, AUTO_INCREMENT	NO	Unique booking reference
user_id	INT	FK → USERS	NO	Who made the booking
show_id	INT	FK → SHOWS	NO	Which show was booked
booked_at	TIMESTAMP	DEFAULT NOW()	NO	When the booking was created
total_amount	DECIMAL(10,2)		NO	Final payable amount after offers
booking_status	VARCHAR(20)	DEFAULT CONFIRMED	NO	CONFIRMED / CANCELLED / PENDING
payment_status	VARCHAR(20)	DEFAULT PAID	NO	PAID / PENDING / REFUNDED

4.9 BOOKING_SEATS (Bridge Table)

Column Name	Data Type	Constraint	Null	Description
booking_seat_id	INT	PK, AUTO_INCREMENT	NO	Surrogate PK
booking_id	INT	FK → BOOKINGS	NO	Parent booking
seat_id	INT	FK → SEATS	NO	Specific seat reserved
seat_price	DECIMAL(8,2)		NO	Price at time of booking (base × multiplier)

4.10 PAYMENTS

Column Name	Data Type	Constraint	Null	Description
payment_id	INT	PK, AUTO_INCREMENT	NO	Unique payment record
booking_id	INT	FK → BOOKINGS, UNIQUE	NO	One payment per booking (1:1)
amount	DECIMAL(10,2)		NO	Amount charged to customer

Column Name	Data Type	Constraint	Null	Description
payment_method	VARCHAR(40)		NO	CARD / UPI / NET_BANKING / WALLET
transaction_id	VARCHAR(100)	UNIQUE	NO	External payment gateway reference
status	VARCHAR(20)		NO	SUCCESS / FAILED / REFUNDED
paid_at	TIMESTAMP	DEFAULT NOW()	NO	Transaction timestamp

4.11 OFFERS

Column Name	Data Type	Constraint	Null	Description
offer_id	INT	PK, AUTO_INCREMENT	NO	Unique offer identifier
offer_code	VARCHAR(30)	UNIQUE	NO	Coupon code e.g. HDFC10
description	VARCHAR(200)		NO	Human-readable offer details
discount_pct	DECIMAL(5,2)		NO	Percentage discount to apply
valid_from	DATE		NO	Offer validity start date
valid_to	DATE		NO	Offer validity end date

4.12 BOOKING_OFFERS (Bridge Table)

Column Name	Data Type	Constraint	Null	Description
id	INT	PK, AUTO_INCREMENT	NO	Surrogate PK
booking_id	INT	FK → BOOKINGS	NO	Which booking used the offer
offer_id	INT	FK → OFFERS	NO	Which offer was applied
discount_applied	DECIMAL(8,2)		NO	Actual INR discount given

5. Foreign Key Relationships

The following table lists all 12 foreign key constraints, the referential actions configured for DELETE and UPDATE, and the business justification for each choice.

FK Column	Child Table	Parent Table	Parent Column	On Delete / Update
city_id	VENUES	CITIES	city_id	RESTRICT / CASCADE — cannot delete city with active venues
venue_id	SCREENS	VENUES	venue_id	CASCADE / CASCADE — deleting venue removes its screens
venue_id	VENUES	CITIES	city_id	RESTRICT / CASCADE — must remove venues before city
screen_id	SHOWS	SCREENS	screen_id	RESTRICT / CASCADE — cannot delete screen with shows
screen_id	SEATS	SCREENS	screen_id	CASCADE / CASCADE — seats belong to screen
movie_id	SHOWS	MOVIES	movie_id	RESTRICT / CASCADE — cannot delete movie with shows
user_id	BOOKINGS	USERS	user_id	RESTRICT / CASCADE — preserve booking history
show_id	BOOKINGS	SHOWS	show_id	RESTRICT / CASCADE — show cannot be deleted if booked
booking_id	BOOKING_SEATS	BOOKINGS	booking_id	CASCADE / CASCADE — booking deletion removes seat records
seat_id	BOOKING_SEATS	SEATS	seat_id	RESTRICT / CASCADE — seat in use cannot be deleted
booking_id	PAYMENTS	BOOKINGS	booking_id	CASCADE / CASCADE — payment tied to booking lifecycle

FK Column	Child Table	Parent Table	Parent Column	On Delete / Update
booking_id	BOOKING_OFFERS	BOOKINGS	booking_id	CASCADE / CASCADE — offer usage tied to booking
offer_id	BOOKING_OFFERS	OFFERS	offer_id	RESTRICT / CASCADE — offer cannot be deleted if used

6. Cardinality & Relationship Types

Each relationship is described by its minimum and maximum participation:

Entity A	Entity B	Cardinality	Description
CITIES	VENUES	1 : Many	One city has many venues
VENUES	SCREENS	1 : Many	One venue contains multiple screens
SCREENS	SEATS	1 : Many	Each screen has a fixed set of seats
SCREENS	SHOWS	1 : Many	One screen runs many shows over time
MOVIES	SHOWS	1 : Many	One movie can have many shows
USERS	BOOKINGS	1 : Many	One user can have many bookings
SHOWS	BOOKINGS	1 : Many	One show can have many bookings
BOOKINGS	SEATS	M : N via BOOKING_SEATS	A booking covers multiple seats; a seat can appear in many bookings across shows
BOOKINGS	PAYMENTS	1 : 1	Exactly one payment per booking (UNIQUE FK)
BOOKINGS	OFFERS	M : N via BOOKING_OFFERS	Multiple offers can be applied to one booking; one offer used in many bookings

7. Normalization Analysis

First Normal Form (1NF): All tables have atomic column values. No repeating groups — seats are stored in a separate SEATS table, not as a comma-separated column in BOOKINGS.

Second Normal Form (2NF): All non-key attributes depend on the whole primary key. BOOKING_SEATS stores seat_price because the price is specific to the (booking, seat) combination — not to either alone.

Third Normal Form (3NF): No transitive dependencies. City information is not repeated in VENUES — a city_id FK references CITIES. Venue address is not repeated in SCREENS — screen_id references SCREENS which references VENUES.

Denormalization exceptions: total_amount in BOOKINGS is a derived value (sum of BOOKING_SEATS prices minus discount). It is intentionally stored to avoid re-computation on every read and to preserve the amount at time of booking in case prices change later.

8. Indexing Strategy

Primary keys are auto-indexed by MySQL. The following additional indexes are created for query performance:

Index Name	Table / Column	Reason
idx_shows_movie	SHOWS.movie_id	Search all shows of a movie — very frequent query
idx_shows_time	SHOWS.show_time	Filter shows by date/time for the listings page
idx_bookings_user	BOOKINGS.user_id	My Bookings page — fetch all bookings for a user
idx_bs_seat	BOOKING_SEATS.seat_id	Check if a specific seat is already booked for a show
idx_payments_txn	PAYMENTS.transaction_id	Payment gateway callback lookup by transaction ID
uq_show_screen_time	SHOWS(screen_id, show_time)	Unique constraint — prevents scheduling two shows at same time on same screen
uq_seat_position	SEATS(screen_id, row, seat_no)	Unique constraint — prevents duplicate seat positions within a screen

9. Key Business Workflows

9.1 Ticket Booking Flow

1. User searches by city + movie → query SHOWS JOIN SCREENS JOIN VENUES JOIN CITIES WHERE movie_id = ? AND city_id = ?
 2. User selects a show → system loads seat layout from SEATS WHERE screen_id = ?
 3. System checks which seats are already taken → subquery on BOOKING_SEATS JOIN BOOKINGS WHERE show_id = ? AND status = CONFIRMED
 4. User selects seats → temporary lock placed in Redis cache (seat_id:show_id key, TTL 10 min)
 5. User confirms + applies offer → INSERT into BOOKINGS and BOOKING_SEATS
-

-
6. Payment gateway processes → INSERT into PAYMENTS with transaction_id
 7. On payment success → UPDATE booking_status = CONFIRMED, payment_status = PAID
 8. On payment failure → UPDATE booking_status = CANCELLED; Redis lock released

9.2 Cancellation & Refund Flow

9. User requests cancellation → UPDATE BOOKINGS SET booking_status = CANCELLED
10. System checks refund eligibility (time before show)
11. Refund processed → UPDATE PAYMENTS SET status = REFUNDED
12. UPDATE BOOKINGS SET payment_status = REFUNDED
13. Seats become available again (BOOKING_SEATS records remain for audit)

10. Sample SQL Queries

Q1 — All open shows for a movie in a city

```
SELECT s.show_id, v.venue_name, sc.screen_type, s.show_time, s.base_price
FROM shows s JOIN screens sc ON sc.screen_id = s.screen_id
JOIN venues v ON v.venue_id = sc.venue_id
WHERE s.movie_id = 1 AND v.city_id = 1 AND s.status = 'OPEN';
```

Q2 — Seats already booked for a specific show

```
SELECT se.row_label, se.seat_number, se.category FROM booking_seats bs
JOIN bookings b ON b.booking_id = bs.booking_id
JOIN seats se ON se.seat_id = bs.seat_id
WHERE b.show_id = 1 AND b.booking_status = 'CONFIRMED';
```

Q3 — Revenue per movie

```
SELECT m.title, COUNT(bk.booking_id) AS bookings, SUM(bk.total_amount) AS
revenue
FROM bookings bk JOIN shows s ON s.show_id = bk.show_id
JOIN movies m ON m.movie_id = s.movie_id
WHERE bk.booking_status = 'CONFIRMED'
GROUP BY m.movie_id ORDER BY revenue DESC;
```

11. Design Decisions & Trade-offs

Price stored at booking time: seat_price in BOOKING_SEATS captures the price at the moment of purchase. This means even if the admin changes base_price or price_multiplier later, historical booking records remain accurate — critical for finance and dispute resolution.

Soft deletes vs. hard deletes: Bookings, payments, and shows use status fields (CANCELLED, REFUNDED) rather than physical deletion. This preserves audit trails and allows business intelligence on cancellation rates. Only venue/screen records without dependencies may be physically deleted (CASCADE behaviour).

BOOKING_SEATS as bridge table: A direct many-to-many between BOOKINGS and SEATS is resolved through BOOKING_SEATS. This bridge table also carries seat_price as an attribute — something impossible with a pure junction approach.

1:1 PAYMENTS relationship: The UNIQUE constraint on payments.booking_id enforces one payment record per booking. If a retry is needed after a failed payment, the existing PAYMENTS record is updated (not duplicated) and transaction_id is replaced.

Composite unique key on SHOWS: UNIQUE(screen_id, show_time) at the database level prevents double-scheduling a screen — even if the application layer has a bug. Defense-in-depth approach.

12. Summary

The BookMyShow database design demonstrates a production-grade relational schema that balances:

- **Integrity:** 12 foreign keys with appropriate ON DELETE/UPDATE actions ensure no orphan records.
- **Performance:** 7 targeted indexes cover all high-traffic query patterns without over-indexing.
- **Scalability:** 3NF normalisation minimises write amplification; denormalised total_amount accelerates reads.
- **Auditability:** Soft deletes and price capture at booking time create a complete financial audit trail.
- **Flexibility:** Bridge tables (BOOKING_SEATS, BOOKING_OFFERS) cleanly handle M:N relationships with additional attributes.

The accompanying SQL file (bookmyshow_database.sql) contains the complete CREATE TABLE statements, all foreign key constraints, indexes, dummy INSERT data for all 12 tables, and 5 sample analytical queries ready to run in MySQL 8.0+.
